

CS571: Programming Languages

Midterm 2 Practice Questions Solution

Subroutines and Control Abstraction

Problem 1 Consider the following code that calculates sum of squares from 0 to n (assuming $n \geq 0$).

```
int SumOfSquares(int n) {  
    if (n == 0)  
        return 0;  
    else  
        return n * n + SumOfSquares(n - 1);  
}
```

1. Explain why this implementation of `SumOfSquares` will take a long time to run and uses a lot of memory (sometimes even run out of memory) with a large n .

Solution: The current implementation requires us to keep activation record/stack frames for every recursive call of `sumsquare` because we will need to combine the result of the recursive call with current instance's n value to get our result. These are not tail-recursive calls. So each round of recursive call, we need to (1) allocate stack frames which takes up lots of memory; (2) complete the calling sequences which maintains the call stack including saving registers, saving return values on stack and so on, which takes a lot of time. If we modify the function to a tail-recursive call, the racket interpreter will save memory by reusing the same activation record and just update parameter values in the same activation record and save time by not needing to go through the whole calling sequences.

2. Rewrite this function `SumOfSquares` into an equivalent tail-recursive function. **Solution:**

```
int SumOfSquares(int n) {  
    return SumOfSquaresHelper(0, n);  
}  
  
int SumOfSquaresHelper(int a, int n) {  
    if (n <= 0)  
        return a;  
    else  
        return SumOfSquaresHelper(a + n * n, n - 1);  
}
```

3. Rewrite the tail-recursive version into an equivalent function using a loop.

Solution:

```
int SumOfSquares(int n) {  
    int a = 0;  
    while (n > 0) {  
        a = a + n*n;  
        n = n - 1;  
    }  
    return a;  
}
```

Problem 2 Consider the following program. Write down what will be printed out when the parameters are passed (1) by value, (2) by reference, and (3) by value return.

```
int x = 5, y = 6;  
void foo(int a, int b) {  
    x = a+b;  
    b = a+a;  
}  
main () {  
    foo(x, y);  
    print(x, y);  
}
```

Solution:

- (1) 11,6 (call by value)
- (2) 11,22 (call by reference)
- (3) 5,10 (call by value return)

Problem 3 What is the output of the following Java program where the exception handling uses “replacemen” semantics?

```
import java.io.*;
class p2 {
    private static void foo(Integer i) {
        try {
            System.out.println("A");
            int j = i.intValue();
            System.out.println("B");
            j = j / 0;
            System.out.println("C");
        }
        catch (ArithmeticException e) {
            System.out.println("D");
        }
        catch (NullPointerException e) {
            System.out.println("E");
        }
        finally {
            System.out.println("F");
        }
    }
    public static void main(String[] args) {
        try {
            System.out.println("G");
            foo(null);
            int x = 461/0;
            System.out.println("H");
        }
        catch (ArithmeticException e) {
            System.out.println("I");
        }
        catch (NullPointerException e) {
            System.out.println("J");
        }
        finally {
            System.out.println("K");
        }
    }
}
```

Solution:

G
A
E

Lambda Calculus

Problem 4 For the following lambda term:

$$T \triangleq (\lambda x \ y. x \ y) \ (\lambda x. y) \ x$$

- a) (4pt) Circle/mark all free variables and connect all bound variables in T to their definitions with lines. For example, the bound variables for $\lambda x. x \ y$ should be $\lambda x. x \ \boxed{y}$.

Solution: $(\lambda x \ \boxed{y}. x \ y) \ (\lambda x. \boxed{y}) \ \boxed{x}$

Notes: To save time during the exam, we do not require you to give details of FV calculation as illustrated below. You only need to correctly use brackets to show bound variables and circles/boxes to identify free variables to receive full points.

$$\begin{aligned}
 FV(T) &= FV((\lambda x. \lambda y. x_1 \ y_1) \ (\lambda x. y_2)) \cup FV(x_2) \\
 &= FV(\lambda x. \lambda y. x_1 \ y_1) \cup FV(\lambda x. y_2) \cup FV(x_2) \\
 &= (FV(\lambda y. x_1 \ y_1) - \{x\}) \cup (FV(y_2) - \{x\}) \cup \{x_2\} \\
 &= ((FV(x_1 \ y_1) - \{y\}) - \{x\}) \cup (\{y_2\} - \{x\}) \cup \{x_2\} \\
 &= ((FV(x_1) \cup FV(y_1) - \{y\}) - \{x\}) \cup \{y_2\} \cup \{x_2\} \\
 &= ((\{x_1\} \cup \{y_1\} - \{y\}) - \{x\}) \cup \{y_2\} \cup \{x_2\} \\
 &= ((\{x_1, y_1\} - \{y\}) - \{x\}) \cup \{y_2\} \cup \{x_2\} && y_1 \text{ is bound to } \lambda y \\
 &= (\{x_1\} - \{x\}) \cup \{y_2\} \cup \{x_2\} && x_1 \text{ is bound to } \lambda x \\
 &= \emptyset \cup \{y_2\} \cup \{x_2\} \\
 &= \{x_2, y_2\}
 \end{aligned}$$

- b) (8pt) Fully evaluate T so that no further β -reduction is possible. Show detailed steps of your computation.

Solution:

$$\begin{aligned}
 &(\lambda x \ y. x \ y) \ (\lambda x. y) \ x \\
 &= (\lambda x. \lambda y. x \ y) \ (\lambda x. y) \ x && (\text{desugar term } \lambda x \ y. x \ y) \\
 &= (\lambda x. \lambda z. x \ z) \ (\lambda x. y) \ x && (\alpha - \text{reduction}) \\
 &= (\lambda z. (\lambda x. y) \ z) \ x && (\beta - \text{reduction}) \\
 &= (\lambda x. y) \ x && (\beta - \text{reduction}) \\
 &= y && (\beta - \text{reduction})
 \end{aligned}$$

Problem 5 Recall the following definitions of Church Encoding we defined in our class,

$$\begin{aligned}
 \underline{n} &\triangleq \lambda f z. f^n z & \underline{0} &\triangleq \lambda f z. z & \underline{1} &\triangleq \lambda f z. f z \\
 \text{SUCC} &\triangleq \lambda n. \lambda f z. f (n f z) & \text{PLUS} &\triangleq \lambda m n. m \text{SUCC } n \\
 \text{PAIR} &\triangleq \lambda x y f. f x y & \text{LEFT} &\triangleq \lambda p. p \lambda x y. x & \text{RIGHT} &\triangleq \lambda p. p \lambda x y. y
 \end{aligned}$$

- a) Fully evaluate $\text{RIGHT } (\text{PAIR } \underline{1} \ \underline{0})$ so that no further β -reduction is possible. Show detailed steps of your computation.

Solution:

$$\begin{aligned}
 &\text{RIGHT } (\text{PAIR } \underline{1} \ \underline{0}) \\
 &= (\lambda p. p \lambda x y. y) (\text{PAIR } \underline{1} \ \underline{0}) && \text{(definition of RIGHT)} \\
 &= (\text{PAIR } \underline{1} \ \underline{0}) (\lambda x y. y) && (\beta - \text{reduction}) \\
 &= ((\lambda x y f. f x y) \ \underline{1} \ \underline{0}) (\lambda x y. y) && \text{(definition of PAIR)} \\
 &= (\lambda f. f \ \underline{1} \ \underline{0}) (\lambda x y. y) && (\beta - \text{reduction}) \\
 &= (\lambda x y. y) \ \underline{1} \ \underline{0} && (\beta - \text{reduction}) \\
 &= \underline{0} && (\beta - \text{reduction})
 \end{aligned}$$

- b) Define a function **MULTPAIR** in λ -calculus so that given a pair of two church numerals, it returns a church numeral that represents the product of the two numbers. For example, $\text{MULTPAIR } (\text{PAIR } \underline{2} \ \underline{3})$ should evaluate to $\underline{6}$ (or some equivalent term under α/β -reduction). You can use all λ -terms above.

Solution: $\text{MULTPAIR} \triangleq \lambda p. \lambda f z. (\text{LEFT } p) (\text{RIGHT } p f) z$